

AI-BASED RESUME SCREENING SYSTEM

Major Project Report

Project Domain	Artificial Intelligence / Machine Learning / Natural Language Processing
Core Technique	Transformer-based semantic embeddings and similarity matching
Application	Automated resume screening and candidate ranking
Interface	Streamlit web application
Primary Technologies	Python, Sentence Transformers, scikit-learn, PyMuPDF, Streamlit

Abstract

This project presents an AI-based resume screening system that compares candidate resumes with a job description using Natural Language Processing and transformer-based sentence embeddings. The system extracts text from PDF resumes, preprocesses the content, generates semantic vector representations, calculates cosine similarity, combines semantic and skill-based scores, ranks candidates, and provides explainable insights such as matched and missing skills. A Streamlit interface is proposed for interactive screening and result visualization. The system is designed as decision-support software and not as an autonomous hiring mechanism.

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Existing System and Proposed System
- 5. System Requirements
- 6. System Architecture
- 7. Methodology
- 8. Algorithms and Scoring
- 9. Implementation
- 10. User Interface and Outputs
- 11. Testing and Evaluation Plan
- 12. Responsible AI and Limitations
- 13. GitHub and LinkedIn Submission
- 14. Future Scope
- 15. Conclusion

1. Introduction

Resume screening is an important stage in recruitment, but manually reviewing a large number of resumes can be slow and inconsistent. Conventional keyword-based screening can also overlook candidates who express similar capabilities using different words. This project explores semantic matching using transformer embeddings so that resumes and job descriptions can be compared according to meaning as well as explicit skills.

2. Problem Statement

Develop an AI-assisted system that accepts a job description and multiple candidate resumes, extracts and processes their content, estimates their relevance using semantic and skill-based matching, ranks candidates, and explains the main factors contributing to each result.

3. Objectives

- Extract text from PDF resumes reliably.
- Normalize and preprocess resume and job-description text.
- Generate transformer-based semantic embeddings.
- Calculate cosine similarity between resumes and job requirements.
- Identify matched and missing technical skills.
- Combine semantic and skill scores into an overall candidate score.
- Rank candidates in descending order of relevance.
- Provide interpretable recommendations and skill-gap information.
- Present results through a simple Streamlit dashboard.
- Document the project for reproducible GitHub publication.

4. Existing System and Proposed System

Aspect	Traditional Keyword Screening	Proposed AI System
Matching	Exact keywords	Semantic + skill matching
Synonyms / wording	Limited	Better semantic handling
Ranking	Keyword count / rules	Weighted relevance score
Explanation	Often limited	Matched and missing skills
Interface	ATS-dependent	Interactive Streamlit dashboard
Extensibility	Rule-heavy	Model and scoring modules can be replaced

5. System Requirements

Software: Python 3.10–3.12, Streamlit, Sentence Transformers, scikit-learn, PyMuPDF, python-docx, Pandas, NumPy and Plotly. Git is recommended for version control.

Hardware: A normal modern laptop is sufficient for a lightweight transformer model. A GPU is optional and can accelerate embedding generation for larger datasets.

6. System Architecture

Stage	Description
Input	Job description and candidate PDF resumes
Document Parsing	Extract text using PyMuPDF
Preprocessing	Normalize whitespace, case and common text variations
Skill Extraction	Detect predefined technical skills
Embedding	Generate semantic vectors with Sentence Transformers
Similarity	Calculate cosine similarity between job and resume vectors
Scoring	Combine semantic and skill scores
Ranking	Sort candidates by final score
Explainability	Show matched skills, missing skills and recommendation
Dashboard	Display rankings, charts and downloadable results

7. Methodology

7.1 Document Processing

The system accepts PDF resumes and extracts text page by page. The extracted content is normalized before semantic comparison while preserving technical tokens that are important for matching.

7.2 NLP Preprocessing

- Convert text to a consistent case.
- Normalize whitespace and line breaks.
- Normalize common variants such as machine-learning and natural-language-processing.
- Retain useful technical characters such as +, #, periods and hyphens.
- Prepare clean text for embedding while retaining source content for explanation.

7.3 Transformer Embeddings

The recommended baseline model is **all-MiniLM-L6-v2**. Sentence Transformers converts a complete resume or job description into a dense vector representation in a shared semantic space. Similar content is expected to have higher vector similarity.

7.4 Skill Extraction

A project-defined skill vocabulary can identify technical terms such as Python, SQL, machine learning, NLP, TensorFlow, PyTorch, Docker, GitHub and cloud platforms. This provides a transparent complement to semantic similarity.

8. Algorithms and Scoring

8.1 Cosine Similarity

For resume vector R and job vector J:

$$\text{Similarity}(\mathbf{R}, \mathbf{J}) = (\mathbf{R} \cdot \mathbf{J}) / (\|\mathbf{R}\| \times \|\mathbf{J}\|)$$

8.2 Final Score

$$\text{Final Score} = 0.70 \times \text{Semantic Score} + 0.30 \times \text{Skill Score}$$

The weights are configurable project parameters. They should not be interpreted as universally optimal weights. For a formal experimental study, they should be tuned against a labelled evaluation set.

8.3 Recommendation Categories

Score	Interpretation
85% and above	Highly / Excellent Match
70%–84.99%	Recommended / Strong Match
55%–69.99%	Needs Review / Moderate Match
Below 55%	Low Match

9. Implementation

The implementation is organized into modular Python files so that document parsing, preprocessing, embeddings, matching, ranking and explainability can be developed independently.

```
AI-Resume-Screening-System/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ src/
■■■ __init__.py
■■■ document_parser.py
■■■ preprocessing.py
■■■ embeddings.py
■■■ matcher.py
■■■ ranking.py
■■■ explainability.py
```

10. User Interface and Outputs

- Job-description text area for entering the target role requirements.
- Multiple-PDF upload control for candidate resumes.
- AI screening button that triggers document processing and scoring.
- Candidate ranking table with rank, semantic score, skill score and final score.
- Candidate score chart for quick visual comparison.
- Top-candidate summary with recommendation category.
- Expandable explainability section showing matched and missing skills.
- CSV export for downstream analysis.

Illustrative Result Format

Rank	Candidate	Semantic	Skill	Final	Recommendation
1	Candidate A	87%	92%	88.5%	Highly Recommended
2	Candidate B	83%	86%	83.9%	Recommended
3	Candidate C	76%	79%	76.9%	Recommended

Note: The values above are illustrative formatting examples, not measured experimental results.

11. Testing and Evaluation Plan

For a formal evaluation, construct a privacy-safe dataset of synthetic or permissioned resumes and job descriptions. Human evaluators can assign relevance labels. The ranking engine can then be assessed using information-retrieval metrics.

Metric	Purpose
Precision@K	Measures how many of the top K candidates are relevant.
Recall@K	Measures how many relevant candidates are retrieved in the top K.
NDCG@K	Measures the quality of the ranking order.
MRR	Measures how quickly the first relevant candidate appears.

Metric	Purpose
Latency	Measures processing time for a batch of resumes.

12. Responsible AI and Limitations

- The system should support human review rather than autonomously making employment decisions.
- Protected attributes and inappropriate personal information should not be used as ranking features.
- Model outputs can contain bias inherited from training data and language patterns.
- A missing detected keyword does not prove that a candidate lacks the underlying capability.
- Similarity scores are not equivalent to hiring suitability or job performance.
- Public repositories should contain synthetic, anonymized or properly licensed sample data only.

13. GitHub and LinkedIn Submission

The GitHub repository should contain the source code, dependency list, README, project report, screenshots and sample data where permitted. The LinkedIn showcase should focus on the working concept, architecture, AI techniques, interface and documented results rather than exposing personal candidate information.

- GitHub repository: AI-Resume-Screening-System
- README: overview, workflow, installation, usage, architecture and limitations.
- Screenshots: dashboard, upload screen, ranking screen and explainability screen.
- Report: final PDF stored under docs/project_report.pdf.
- LinkedIn: project overview, technologies, key features and repository reference.
- Submission image/video: show code structure and application/result interface where required.

14. Future Scope

- Fine-tune a domain-specific embedding or reranking model.
- Add multilingual resume support.
- Support OCR for scanned resumes.
- Introduce a structured skill ontology.
- Add cross-encoder reranking for top candidates.
- Add configurable mandatory requirements.
- Add bias and fairness evaluation.
- Deploy using Docker and a cloud platform.

15. Conclusion

The AI-Based Resume Screening System demonstrates how NLP and transformer-based semantic representations can be integrated into an end-to-end recruitment decision-support application. The project combines document processing, preprocessing, embeddings, similarity scoring, skill matching, ranking and explainability in a modular architecture. Its major-project value comes from integrating these components into a usable Streamlit interface and from emphasizing evaluation and responsible AI. Future work can improve domain adaptation, multilingual processing, fairness analysis and deployment scalability.

Keywords: Artificial Intelligence, Machine Learning, NLP, Resume Screening, Sentence Transformers, Semantic Similarity, Candidate Ranking, Explainable AI, Streamlit